

Trajectory optimization with inverse dynamics problem via neural networks

1st Ilia Milyoshin

Innopolis University

Innopolis, Russia

i.milyoshin@innopolis.university

2nd Anton Kirilin

Innopolis University

Innopolis, Russia

a.kirilin@innopolis.university

3rd Lev Kozlov

Innopolis University

Innopolis, Russia

l.kozlov@innopolis.university

models training, architecture construction analysis, dataset preprocessing, documentation simulation and optimization

Abstract—With the development of artificial intelligence technologies, the use of neural networks has become commonplace for all fields of science. Although mathematics and statistics are the sciences on which most machine learning methods are based, control theory, which is part of mathematics, can use neural networks to solve some problems in this area. Determining the physical characteristics of a system using AI can greatly simplify and speed up the process of simulation and subsequent control of robots. Moreover, real-world systems often have some uncertainties in parameters, which complicate the design of a robust algorithm. On the other side training AI models relies on the training dataset. Models trained on artificial data perform bad on the real-world data due to the noise which came as a result of imperfection of sensors and non-accountable factors such as viscose friction, power loss, external forces acting. In this study we aim to resolve such issues with adoption of noise-prone techniques from computer vision field. In addition, we prove that denoising-NN performer better on a noisy data in comparison with ordinary ones which suggests that this technique should be used for real-life control algorithms.

Index Terms—Control theory, system identification, neural networks, trajectory optimization

I. INTRODUCTION

In this paper we address the problem of system identification. Most of the control approaches require a knowledge about the system. However, these system are not always linear and easy to construct, therefore control engineers have to spend more time on describing the system by writing the forward and inverse dynamics. As a solution for this problem we propose a similar approach to [1]. We would like to use data collected from the robot then train a neural network that would solve the inverse dynamics problem for a specific system with unknown parameters. After that the pretrained model is used in the trajectory planing as a system descriptor.

II. LITERATURE OVERVIEW

III. MATHEMATICAL FOUNDATION

A. Trajectory planing

Although there are different approaches for trajectory optimization with specific pros and cons, we have considered our possibilities and chosen the following idea:

$$\min_{\mathbf{q}_i, \dot{\mathbf{q}}_i, \ddot{\mathbf{q}}_i} g(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}})$$

subject to

$$\mathbf{q}_{i+1} = \mathbf{q}_i + \dot{\mathbf{q}}_{i+1} \cdot dt$$

$$\dot{\mathbf{q}}_{i+1} = \dot{\mathbf{q}}_i + \ddot{\mathbf{q}}_i \cdot dt$$

$$\mathbf{u}_i = f^{-1}(\mathbf{q}_i, \dot{\mathbf{q}}_i, \ddot{\mathbf{q}}_i)$$

$$\mathbf{u}_i \in U$$

The formulation might be a bit confusing outside of the robotics field and there might be some inaccuracies, but let us explain it in the words.

We want our optimization to propose a set of states, namely, position, velocity and acceleration of dynamic model such that each step is dependent on previous using Euler integration scheme. Moreover, we add constraint on dynamic feasibility: for dynamics to experience required acceleration there should be enough energy from actuators, and, usually, our control input is limited.

In the view of the whole project we aimed to train Neural Network to determine the torque needed by model to satisfy some trajectory and constrain the optimization task from unfeasible solutions.

B. Data denoising

Modeling real-world systems is based on simplification of all processes that can happen. There is a trade-off between the complexity of a physical model and its accuracy. Usually relatively small forces, variations of mass, mutual influence of control signals are not take into account when the systems are designed. This leads to an error between the desired behavior of the system and its' actual response. In certain areas the error is not significant and the speed of computation is more desirable e.g. conveyor manipulator, whereas for robots designed for high precision tasks space (e.g. surgical robots) a slow, but real-world accurate models are more desirable.

The more complex the model the harder to design it. As we aimed to simplify the process of solving the inverse dynamics problem we also tried to seek for a method that could work with noisy data, where the noise is representing the factors that are not taking account. Training a neural network on noisy input and non-noised output is a technique that is commonly used in computer vision fields. Diffusion models is an example

of a noise-based generative algorithm are able to generate images from a complete noise by learning the distribution of the training dataset (e.g. images of airplanes).

In our case we assume that for the same mechanisms there is a function that maps generalized components into a distribution of torques. These distributions follow the same type, have a little to none variation in standard deviation and the only difference lies in their mean values. We make this assumption due to the fact that non-accountable forces usually do not interfere with the commonly-used control techniques, which shows that they are accountable only for a small deviation of the final prediction.

IV. DATA COLLECTION

A. General dataset generation process

We collected the data for training the models using mujoco simulation. This is because we have no hardware equipment for conducting this in real life. Moreover, our main focus was on proving the fact that it is possible to train a neural network as a function for inverse dynamics of a system and access the new method of dealing with imperfection of the simplified modeling, but assembling the hardware system would took our time and would not give any significant benefit other than proving our theory. For this project we did several non-complex, but chaotic systems such as double pendulum and cart pole.

For each model type we used a specific urdf file, so the physical characteristics of a system are the same, however all experiments had different initial positions. This altogether with the fact that the systems are chaotic gives us distinctive datasets from statistical point of view, but similar from physical. The approach with real-world solution will be the same, since we are not focusing on time-dependent systems, that change their properties with time. However it's possible to construct a model that is deterministic with respect to time, in this case you have to check that time-specific event occur at the same time-stamp.

B. Noise generation

As we discussed earlier we assume that the noise occurs due to the simplification of the real-world systems. Which is why we suggest two different techniques in noise generation.

1) *Artificial Gaussian noise*: After generating the initial dataset we can add a normal distribution with mean value $\mu = 0$ and standard deviation $\sigma = \mu_\tau \frac{1}{100}$. It is important not to overnoise the initial data otherwise the model will not be able to learn the physics of the mechanism. However, this is an artificial noise, which only mimics the real distribution that we aim to learn. Nonetheless, this noise is easier to implement and easier for a model to learn, which leads to a better denosing correction on the same dataset.

2) *Friction*: The process of data generation is basically a simulation of a system with random control input. We basically reverse the dataset for usage in the inverse dynamics problem. Mujoco simulator allows to specify viscous friction coefficient. Since the control law is random we constantly adding the

energy to the system and the system does not converge to a stable position which would affect the distribution of the dataset and lead to inaccurate models.

V. NOISELESS MODEL

A. Future state prediction

Initially we proposed to use multiple previous states to predict the future state. However, this does not match our project proposal. In this scenario the model is responsible not only for system identification, but also for integrating the state. We have tried fully connected neural networks(FCNN), long short-term memory(LSTM) models, but the accuracy of them was lower than 30%. This was due to the fact that FCNN tend to overfit a specific experiment scenario and LSTM cannot learn any generic patterns at all (fig. 1(a)-1(b)).

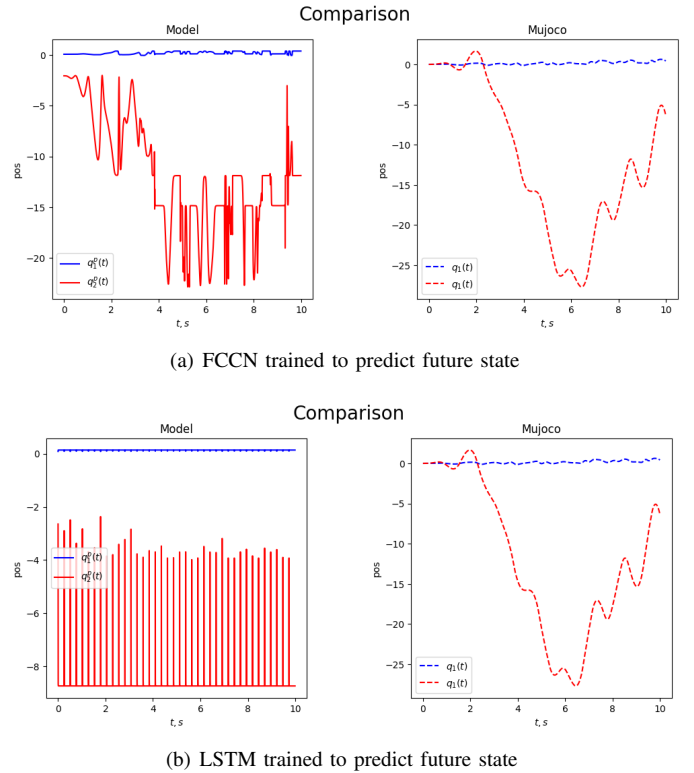


Fig. 1: Models predicting future state

B. Generalization of data

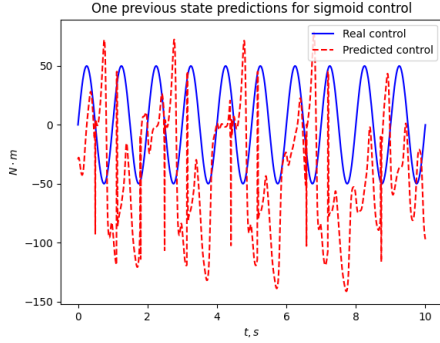
For the inverse-dynamics problem we started with training a model on a specific experiment and tested it on others. However, in this case model performed purely. Because of this problem we decided to concatenate all the data for one physical system into one big dataset and then split it into train, test and validation. However, this also requires to change the method of describing the systems, because we have to take consequent states at each step. This would either make things less general or more complex, so we decided also to use the inverse dynamics problem and after that we can combine different experiments.

C. Inverse dynamics problem

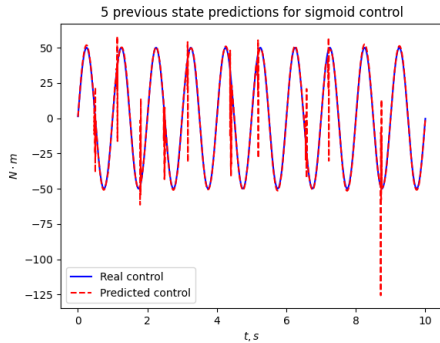
As we stated earlier trajectory optimization algorithms require a function of inverse dynamics to calculate torques and forces. For this purposes we created two python classes that are working with the data from the MuJoCo [2] simulations, trained on it distinctive models and analyzed them. For the same system parameters, but different experiments we concatenated all the data. Therefore the training process uses generalised data to get the info about physical properties of a robot.

D. Testing on validation sets

We trained a recurrent neural network (RNN) and FCNN in corresponsing order. We tested both models on the validation sets from experimental data to see if it does learn general properties of a system. RNN required more cycles in order to make correct predictions (fig. 2(a)-2(b)). On the other hand a simple FCNN with only 3 layers performed well on all systems validation datasets (fig. 3(a)-3(b)).



(a) 1 state RNN



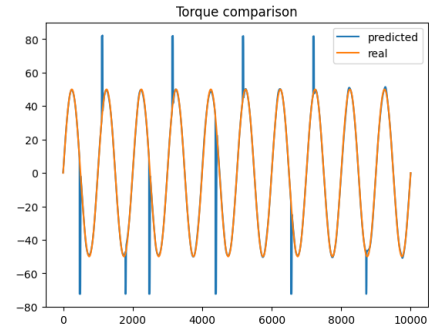
(b) 5 state RNN

Fig. 2: RNN models

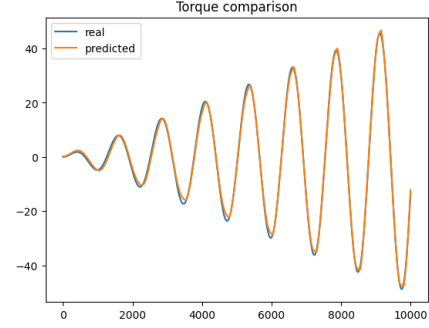
E. Trajectory optimization

Our trajectory optimization technique requires to know the function of system's inverse dynamics.

$$\mathbf{u}_i = f^{-1}(\mathbf{q}_i, \dot{\mathbf{q}}_i, \ddot{\mathbf{q}}_i)$$



(a)



(b)

Fig. 3: FCNN predictions

All the work done with the machine learning models was to approximate this function without knowing the details of the physics under the hood. Now we have to transform the trained neural network to a parametric function that can be understood by casadi framework. Due recurrent to complexity of RNN that slows down the process we decided to use 3-layer FCNN.

After performing the whole process of optimization we have to compare it to other simulators. In our case we took pinocchio library and compared results of integration with our approach and Articulated-Body Algorithm (aba) of the aforementioned library. The results showed that the error between 2 approaches is less than 0.01 unit of measure (fig. 4(a)-4(b)).

VI. DENOSING MODEL

After conducting experiments on replicating the behavior on noiseless synthetic data we have decided to experiment with denoising techniques. As we stated earlier the easier approach is to add a specific Gaussian noise to already gathered data without the change in the target, which forces the model to learn a distribution mapping function rather than a standalone many to one mapper.

For this experiment we did not aim to create a probabilistic, the goal was to find an architecture that is more accurate on solving the inverse dynamics problem with noisy dataset. Because the system is physical the real-world models often use predefined mathematical functions in the final solution. Under this fact we made an assumption that experimenting

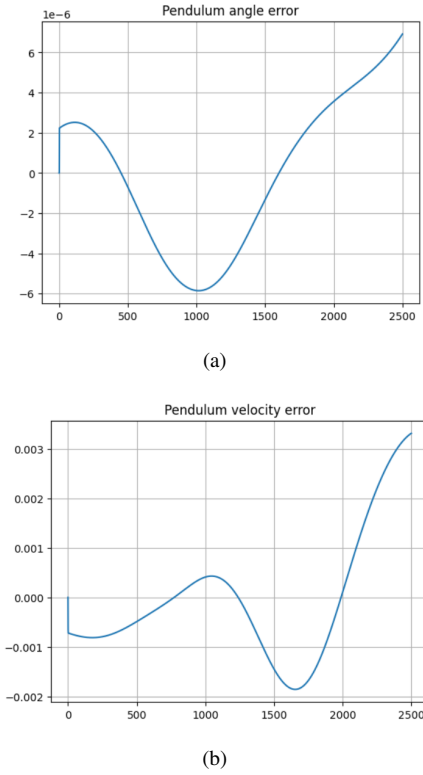


Fig. 4: Error of our optimization compared to library method

with activation functions is also suitable for defining the architecture of our model. That is why we have tried out different activation function in the FCNN model such as sine function, square function, and $f(x) = x \sin(x)$ function. In a sense this is not an activation function rather a transformation applied to the data. In order to prevent the hard definition those function were combined into a whole one SemiAct function that takes as an input full output of the previous layer and returns a vector where to each element was applied only one transformation or none (figure ??).

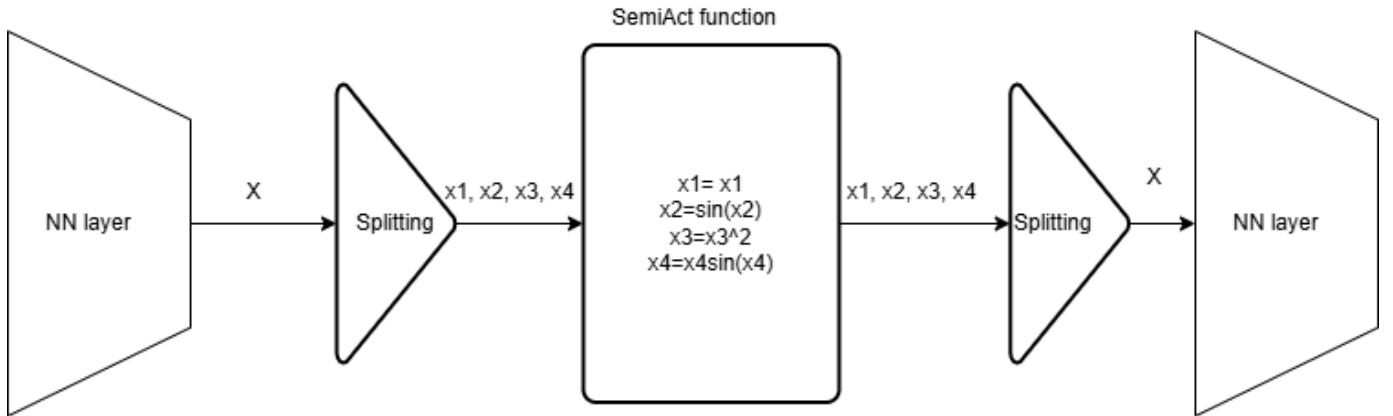


Fig. 5: SemiAct function

VII. CONCLUSION

The results show that neural networks can be a helpful tool in dealing with systems with unknown physical laws and/or when it is easier to collect the data and to train a simple FCNN model rather than to compose the equation from scratch. However, this method increases the optimization process to a complex conversion of a neural network to a parametric function in optimizing libraries like casadi. Moreover, complex mathematically by definition models like Recurrent neural networks may not only be hard to convert, but also produce bad quality results on lower numbers of iterations. One can find our work artifacts in this GitHub repository: [link](#)

For future works we suggest to:

- To experiment with other ML algorithms that are not too hard to parameterize
- To test other physical systems, that are more chaotic
- Assemble the experiment in hardware and prove our hypothesis that this method can help to deal with real-life black-box systems.

REFERENCES

- [1] Lennart Ljung, Carl Andersson, Koen Tiels, and Thomas B. Schön. Deep learning and system identification. *IFAC-PapersOnLine*, 53(2):1175–1181, 2020. 21st IFAC World Congress.
- [2] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, 2012.